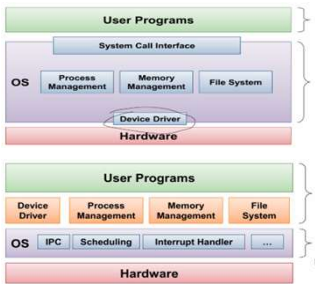
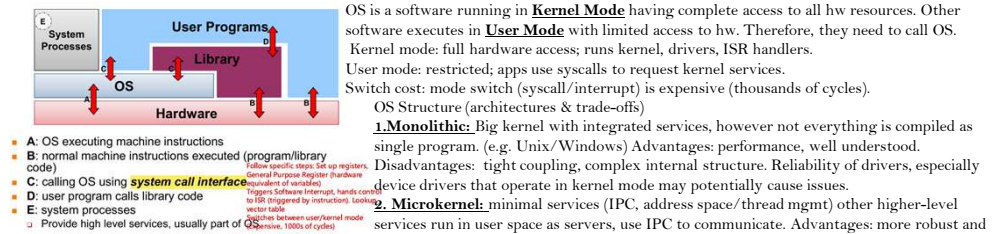


Lecture 1: Introduction to OS

a program that acts as an intermediary between users (and applications) and the hardware; provides abstraction, resource allocation and control/protection. **3 core roles:**

- 1. Abstraction:** (Hides hw differences and exposes stable high-level APIs (files, processes, sockets). User perform tasks through OS (efficient, portable)
 - 2. Resource Allocation:** Manages CPU, memory, I/O: multiplexes/schedules to maximize utilization and fairness. (multiple programs simultaneously)
 - 3. Control/Protection:** enforces isolation, prevents misuse, handles fault, security/access control (e.g. Prevents processes from accessing data from other processes, ensures separate user spaces, prevents programs from misusing computer). Historical Progression: No OS -> Batch OS -> Time-sharing -> minicomputer/PCs. OS Services: Process Management, Threads, Memory management, File system, I/O & Drivers, Security, System Call Interface
- Role of ISR in OS:** How does computer hand control to OS? via ISR. E.g. every 1ms, ISR hands control to OS, or OS is booted up by an ISR vs keyboard interrupt (on button) How are segmentation faults detected? ISR

OS Structure: How does OS implement functionality to fulfill its specifications? Organize components for flexibility, robustness, maintainability.



Lecture 2: Process Abstraction

What is a Process: Program that has been loaded into memory with 3 steps (1. Load program into memory, 2. OS creates 3 context (hardware, memory, os) 3. create entry in Process Table known as Process Control Block). It is a dynamic abstraction of a running program – everything the CPU needs to suspend/resume it. Allows for sharing of hardware for efficient utilization. (Diff programs, diff regions in memory, diff programs change CPU state, prevent programs from interfering with one another)

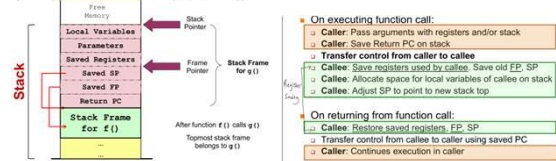
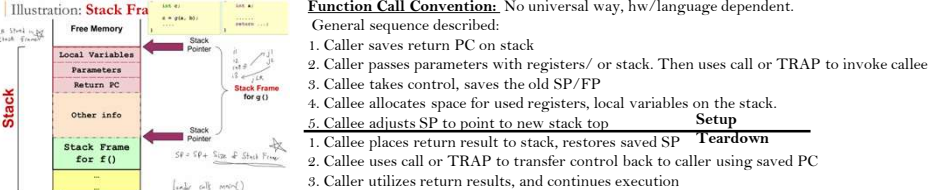
Process Abstraction: Information describing a running program, or **Context:** 3 types **1. Memory Context:** Text (code), Data (global/static variables), Stack (function calls, local variables, return addresses), Heap (dynamic allocations).

2. Hardware Context: PU state needed to resume execution: Program Counter (PC), General Purpose Registers (GPRs), Stack Pointer (SP), frame pointer (FP), special registers. **3. OS Context:** OS bookkeeping: PID, process state, open files, accounting, privileges, pointers to memory regions, scheduling info (priority, timeslice).

Recap on **Computer Organisation:** Fetch (Cache memory that allows for faster access, location controlled by PC), Decode, Execute (functional units), Memory, Writeback (Registers: GPR, PC, SP, FP, Program Status Word (PSW)) cycle. Note that Program Counter is address of current instruction in text memory, SP is address of top (next free spot) of stack memory.

Function Calling: Issues with using data memory space for local variables: 1. Scope, 2. Lifetime 3. Return. Control Flow issues: Jumping to function body and returning after function call is done (Store SP). Data Storage issues: passing params to function, capture return result, local variables.

Stack Memory Region: Stores information related to function invocation in a Stack Frame. **Stack Pointer (SP):** points to top (first unused). SP changes on call/return → hard to use as stable anchor. **Frame Pointer (FP):** optional register pointing to fixed offset inside current frame; makes accessing locals/args predictable. Goes from low → high addresses upwards.



Function Call Convention: No universal way, hw/language dependent. General sequence described:

1. Caller saves return PC on stack
2. Caller passes parameters with registers/ or stack. Then uses call or TRAP to invoke callee
3. Callee takes control, saves the old SP/FP
4. Callee allocates space for used registers, local variables on the stack.
5. Callee adjusts SP to point to new stack top
6. Callee places return result to stack, restores saved SP
7. Callee uses call or TRAP to transfer control back to caller using saved PC
8. Caller utilizes return results, and continues execution

Tutorial 1 Call sequence: 1. Caller (setup) Save caller \$fp and \$sp on stack. 2. Copy \$sp → \$fp (so \$fp is stable anchor). 3. Reserve space for args + saved \$ra + saved regs: addi \$sp,\$sp,N. 4. Store arguments at offsets from \$fp (e.g., sw \$a, 8(\$fp) etc.).jal callee. Callee 1. Save \$ra (and any callee-saved registers it will use) into the frame (e.g., sw \$ra, 16(\$fp)). 2. Optionally allocate more local space (addi \$sp,\$sp,M) and save temporaries. 3. Access parameters using \$fp offsets, compute, write result back to a known offset (e.g., 8(\$fp)). 4. Restore saved regs and \$ra from stack. 5. jr \$ra to return. 6. Caller retrieves result from stack, restores \$sp and \$fp.

Heap (Dynamic Allocation): Heap allocations occur at runtime and are freed unpredictably → cannot use static Data or LIFO Stack. E.g. C malloc(), C++/Java new, Heap problems to know: **Fragmentation:** freed blocks create holes (external fragmentation) → allocator strategies: first-fit, best-fit, buddy system, slab allocators. **Memory leaks:** failure to free → long-lived memory growth. **Concurrency:** allocator must be thread-safe (locks per arena, per-thread caches).

Process ID and State: PID distinguishes processes (reserved PID 0 in unix for init) PID reuse, max. no. of processes are OS dependent. Processes can be ready to run, but not running because: Another process / OS is running. No. Processes > No. Functional Units.

Process State Transitions: Create (nil->New): New process created. **Admit(New->Ready):** Process ready to be scheduled. **Switch(Ready->Running):** Process selected to run. **Switch(Running->Ready):** Gives up CPU or is pre-empted on Timer basis. **Event Wait(Running->Blocked):** Waiting for resource/service that is not available yet e.g. syscall or waiting for I/O. **Event Occurs(Blocked->Ready):** Event occurs with Interrupt, process continues. **Exit(Running->Terminated):** Queuing model: **Ready queue, Blocked queue.** With n processes and m CPUs, ≤ m running at once. Scheduler picks from ready queue based on algorithm.

Process Table and Process Control Block (PCB): Minimal/typical PCB fields: PID, parent PID, Process state, Program counter (PC), general registers (GPRs) snapshot, SP and FP, Memory map descriptors (base/limit or page tables): pointers to text/data/stack/heap/info, Open file descriptors, directory info, Scheduling info: priority, timeslice remaining, CPU usage stats, Accounting info, credentials/ security context, Pointers to kernel stacks, kernel data structures (for fast access) The OS stores one PCB per process in a process table (array/hash/list).

System Calls: API to OS. For user programs to request kernel services (I/O, Process control, file ops) requires a change from user to kernel mode (Context switch)

Typical mechanism: 1. User calls a library wrapper (e.g., printf, open) which prepares args and places syscall number in designated register or stack. 2. Execute a special instruction (TRAP, syscall, svc, int 0x80) to switch CPU from user mode → kernel mode. 3. Kernel dispatcher inspects syscall number and jumps to appropriate syscall handler (vectored table). 4. Handler executes, possibly blocking the process (e.g., I/O) or performing work and returning a result. 5. Kernel returns to user mode; library wrapper returns to application. E.g. getpid(), read(), write().

Exceptions vs Interrupts: Exception (synchronous): caused by the currently executing instruction (divide by zero, illegal memory access, page fault). Handler runs immediately; often the instruction is retried or the process terminated. **Interrupt (asynchronous):** external hardware event (timer, keyboard, network). Typically invokes ISR / bottom half; may wake blocked processes. Handler responsibilities (both): save CPU state, handle event, restore state (or change PC) and return. For interrupts, kernel often defers lengthy work to a lower-priority context (softirq, bottom halves).

Tut2: Process Abstraction: fork() creates new process by duplicating calling process. The parent fork() returns PID of child while child fork() returns 0. Child process has exact copy of parents memory (data, stack, heap) at moment of fork(). Each fork() multiplies number of processes, draw process tree to track rs. fork() does not impose execution order, each process is scheduled independently by OS. (Ordering is nondeterministic unless you synchronize). Global invariants: things that happen before first fork(). Use of sleep() affecting ordering, when child is paused, siblings or parent can take control, it biases scheduling but still does not guarantee order. wait() blocks calling process until any child terminates. Using loop with wait(NULL) to wait for N children ensures that parent will only continue after N terminations. waitpid(pid, NULL, 0) blocks until specific child with pid terminates, this can force parent to wait on children with w a particular order. exec(“./PF”, “PF”, arg, NULL) replaces the child image with a program PF (worker process runs independent program)

Lecture 3: Process Scheduling

Concurrent Execution: Virtual (timeslicing) / Physical (multiple CPUs) parallelism. Scheduling problem: when there are more ready processes than CPUs, decide which to run next. The OS component that makes this decision is the **scheduler** and the rule it follows is the **scheduling algorithm**.

Core vs CPU: **Core** contains its own functional units, control units, memory unit, cache, may be on same die and share connections. However, the CPU has all that and its own connections (i.e. I/O ports)

2 Types of Scheduling Policy: 1. **Non-preemptive:** running process stays on CPU until it blocks or yields. 2. **Preemptive:** running process can be interrupted (usually at timer interrupts) and replaced. Preemptive is needed for good responsiveness in interactive systems. **Timer interrupt interval (ITI):** hardware timer frequency (often 1–10 ms). **Time quantum:** time slice given to a process (commonly 5–100 ms). Quantum must be multiple of ITI. Scheduling steps: Scheduler triggered, OS takes over. Context switch and moves process into blocked, queue, pick suitable process P to run based on scheduling algo, setup context of P, let P run.

Response time: StartTime – ArrivalTime. Turnaround Time: CompletionTime – ArrivalTime. Throughput: Number of completed jobs / Time interval

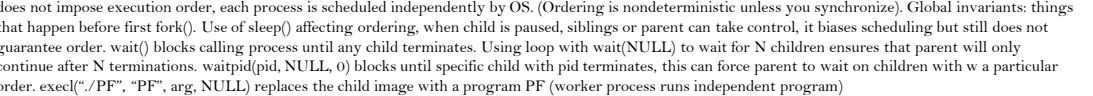
Interactive Scheduling Algorithm: Round Robin (RR): Queue: FIFO ready queue; run head for one quantum; on expiry put it to queue tail. Preemptive. Guarantees an upper bound on wait: with n tasks and quantum q, a task waits at most (n–1)q before getting CPU. Good for fairness & responsiveness; quantum tuning critical. Choice of time quantum: Big quantum has better CPU utilization but longer wait times, Small quantum has bigger overhead but shorter wait.

Priority Scheduling: Assign priority values; always run the highest priority ready task. Can be both preemptive or non preemptive. Starvation of lower priority tasks. Mitigation: aging of priority (gradually raise priority of waiting tasks), time quotas, priority decay (MLFQ).

Scheduling for Real-Time Systems: Tasks are periodic and characterized by Ti (period, how often job arrives) and Ci (Execution time, use WCET if unknown) deadlines: Usually equal to the next arrival (deadline=period) in simplest model, calculation must be completed before next period to prevent data loss (for a task running at nPi, deadline is (n+1)Pi). Operate on CPU cycles = running time/time per cycle. **Fixed Priority:** each task manually assigned priority, with higher priority tasks picked first, and preempting any lower priority tasks. Question: How is this priority assigned? Generally, in RT Systems, all tasks Start at Boot, Last Lifetime of program, end at Shutdown.

Schedulability: If we can schedule a series of tasks so that they can all meet their deadlines, we say that they are schedulable.

Generic 5-State Process Model



Rate Monotonic Scheduling (RMS): assigns static priorities based on period: shorter period → higher priority (optimal among fixed-priority schemes for deadlines = periods). Optimal assignment of priorities.
Testing Schedulability of RMS: Unroll to get Hyper-Period. Simulate the schedule over the hyper-period (LCM of all periods). If every job in that interval meets its deadline, the system repeats and is schedulable. Can be tedious if LCM is large.

1. CPU Utilization $U = \sum \frac{C_i}{T_i}$ has to be less than or equals to 1, which is a necessary but not sufficient condition of schedulability.
2. Liu and Leyland Bound $U \leq n(2^{\frac{1}{n}} - 1)$ is a sufficient but not necessary condition. Thus utilization may not be within bound but still schedulable
3. Critical Instance Analysis: For fixed-priority (RMS) the critical instant often occurs when all tasks release simultaneously (synchronous release at t=0). Simulate the busy period that starts at that instant and check if any job misses.

■ **RMS Example 1:**

Critical Instance Analysis Test 3!

1. Sort T by period of each task, if T is not already sorted.
We will assume that T₁ has the shortest period, T₂ has the 2nd shortest, etc.

2. For each task T_i ∈ T, recursively compute S₁₀, S₁₁, ... where:
- a. $S_{i,0} = \sum_{j=1}^i C_j$

b. $S_{i(x+1)} = C_i + \sum_{j=1}^{i-1} C_j \times \left\lceil \frac{S_{i,x}}{P_j} \right\rceil$

$T_1 : C_{1,0} = 1 < P_1(4)$

$C_{2,0} = 1 + 2 = 3 < P_2(6)$ (includes being preempted by T₁)

$C_{2,1} = 2 + \left\lceil \frac{3}{4} \right\rceil \times 1 = 3$ (includes being preempted by T₁ and T₂)

T ₁	4	1
T ₂	6	2
T ₃	8	2
- $S_{2,0} = 1 + 2 + 2 = 5$

$S_{2,1} = 2 + \left\lceil \frac{5}{4} \right\rceil \times 1 + \left\lceil \frac{5}{6} \right\rceil \times 2 = 2 + 2 + 2 = 6$

$S_{2,2} = 2 + \left\lceil \frac{6}{4} \right\rceil \times 1 + \left\lceil \frac{6}{6} \right\rceil \times 2 = 2 + 2 + 2 = 6$

$6 < 8$
- | Task | Pi | Ci |
|------|----|----|
| T1 | 6 | 2 |
| T2 | 8 | 2 |
| T3 | 4 | 1 |

$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_j^{(k)}}{T_j} \right\rceil C_j$
- T2 is schedulable.

□ T3 and T1 are the same as before, so we will only work on T2.

Stop when S_{i,(x+1)} = S_{i,x}. Call this S_{i,(x+1)} the final value S_{i,F}. I.e. let S_{i,F} = S_{i,(x+1)} when the iteration terminates.

3. If S_{i,F} < D_i, then the task i is schedulable and will not miss its deadlines.

Earliest Deadline First(EDF): EDF runs job with earliest absolute deadline next. Optimal scheduling policy for uniprocessor preemptive systems. Schedulability is up to full CPU utilization of 1. If U <= 1, tasks are guaranteed to be schedulable, however impractical in real world because of cost at every cycle for the OS to sort all tasks by earliest deadline.

B(1)=1.0	B(4)=0.756	B(7)=0.728
B(2)=0.828	B(5)=0.743	B(8)=0.724
B(3)=0.779	B(6)=0.734	U(x)=0.693

2 — Simple formal model (useful for reasoning)

- Let Thread A have ordered events A₁, A₂, ..., A_m.
- Let Thread B have ordered events B₁, B₂, ..., B_n.
- A valid interleaving is any sequence of length m + n that contains all A_i and B_j while preserving the order of the A_s among themselves and the order of the B_s among themselves.

Counting interleavings: number of valid interleavings = $\binom{m+n}{m}$. (Reason: choose the positions for the m A-events among the m + n slots; the rest are B-events.)

Example (step-by-step):

- If m = 2 and n = 2, count = $\binom{4}{2}$.
 - Compute $\binom{4}{2} = \frac{4!}{2!2!} = \frac{24}{2 \cdot 2} = \frac{24}{4} = 6$.
- Those six interleavings are exactly the 6 sequences you can enumerate while keeping each thread's event order.

3 — Why interleavings cause bugs (the read-modify-write pattern)

Many concurrent bugs come from a Read-Modify-Write (RMW) operation implemented as separate steps (read then write). Consider a shared variable x and two threads:

Thread A: x ← x + 2 → modeled as: LA (load x), compute, SA (store).

Thread B: x ← x + 3 → LB then SB.

Each thread must do its own load before its store, but the four operations LA, SA, LB, SB can interleave. Different interleavings produce different final x values. We enumerated these in class: with starting x = 3 the possible outcomes are {1, 3, 7, 9}. That is exactly an interleaving problem.

Worked-through example (one interleaving):

- Interleaving: LA, SA, LB, SB
 - LA reads 3 → A computes 1 → SA writes 1.
 - LB reads 1 → B computes 3 → SB writes 3.
 - Final x = 3.

Do this for each interleaving to get the full set of outcomes.

Lecture 4: Threads

Motivation: Processes are heavy. Creating a process (fork) duplicates address space and OS bookkeeping; context switching between processes saves/restores many things and may flush caches/TLBs. Due to independent memory spaces, difficult to communicate between processes (IPC).
Threads are lighter-weight units of execution inside a process that share most resources but have independent CPU state (registers, PC, stack).
Threads of control: Traditional processes have a single thread of control where only one instruction of the whole program is executing at once, but when we add more threads of control, multiple parts can execute simultaneously.
Single process can have Multiple Threads. Shared: Code (text), global/static data, heap (allocated memory), open file descriptors, process id and OS resource descriptors. **Per-thread:** Thread ID (TID), program counter (PC), general purpose registers, stack (private stack memory), thread-local storage (TLS) if used, and scheduling state (priority, timeslice)
Sharing memory makes communication easy (no IPC), but also exposes shared mutable state problems (races) that must be synchronized.
Threading Models

Model	Pros	Cons
User-level threads (many-to-one from user library): Threads implemented in a user-space library; kernel only sees the process. Thread operations (create, yield, schedule) are library calls, not syscalls.	Very fast create/switch (no syscall), highly configurable scheduling policies inside the process, work on any OS.	Kernel doesn't know about threads — if one thread makes a blocking syscall the entire process blocks, so all threads block; cannot exploit multiple CPUs (single kernel thread)
Kernel-level threads (one-to-one scheduling by kernel) Kernel knows individual threads and schedules them separately (each thread looks like a schedulable entity). Thread Table used.	True parallelism on multi-CPU systems; blocking syscalls block only the calling thread; kernel-level scheduling/policies available.	Thread operations are syscalls → slower; more kernel data structures per thread (more memory/overhead).
Hybrid (many-to-many or many-to-one mapped to kernel threads) User threads mapped to a (smaller or equal) number of kernel threads. Example: Solaris threading models. Hybrid tries to combine flexibility with parallelism.	OS Schedules kernel threads only, and user threads bind to these kernel threads. If a kernel thread is blocked, user threads bind to it are affected while others are not.	

Hardware support: Simultaneous Multithreading: Modern CPUs sometimes provide hardware threads (SMT / Hyper-Threading): multiple logical contexts share core resources (fetch/dispatch/ALUs). This lets a single core run multiple threads more efficiently when one thread stalls (e.g., cache miss). The OS still schedules at the thread level but the CPU provides faster context switching and more register sets in some designs.
POSIX threads pthreads: Standard API on Unix-like systems, can be implemented as user or kernel threads.

Pthread Creation

- Thread creation syntax:**

```
int pthread_create(  
    pthread_t *tidCreated,  
    const pthread_attr_t *threadAttributes,  
    void* (*startRoutine) (void*),  
    void *argForStartRoutine );
```
- Returns:**
 - 0 to indicate success; != 0 for errors
- Parameters:**
 - tidCreated: Thread Id for the created thread
 - threadAttributes: Control the behavior of the new thread
 - startRoutine: Function pointer to the function to be executed by thread
 - argForStartRoutine: Arguments for the startRoutine function

Tut4 pthread:
pthread_create spawns a thread. attr may be NULL for defaults; startRoutine receives a single void * argument.
pthread_exit ends the calling thread and sets the value returned to pthread_join. Returning from the thread function is equivalent to pthread_exit(return_value).
pthread_join waits for the specified thread and optionally receives its return value (a void *).
pthread_detach marks the thread to auto-reclaim resources at exit (cannot join a detached thread).
Mutex idea: A mutex serializes access to a critical section — only one thread holds the mutex at a time. q4 wraps sum++ with a pthread_mutex_lock / pthread_mutex_unlock pair; the final printed sum is reliably 100000.
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // or pthread_mutex_init(&mutex,NULL);
pthread_mutex_lock(&mutex);
sum++;
pthread_mutex_unlock(&mutex);

Pthread Termination

- Thread termination syntax:**

```
void pthread_exit( void* exitValue
```
- Parameters:**
 - exitValue: Value to be returned to whoever synchronizes this thread (more later)
- If pthread_exit() is not used, a pthread will terminate automatically when the end of the startRoutine is reached
- In this case, there is no way to return a exit value

Pthread: Simple Synchronization

- To wait for the termination of another pthread:**

```
int pthread_join( pthread_t threadID,  
    void ** status);
```
- Returns:**
 - 0 to indicate success; != 0 for errors
- Parameters:**
 - threadID: TID of the pthread to wait for
 - status: Exit value returned by the target pthread